

For emulator issues : (change shivli to student)

```
& "C:\Users\shivli\AppData\Local\Android\Sdk\platform-tools\adb.exe" kill-server  
& "C:\Users\shivli\AppData\Local\Android\Sdk\platform-tools\adb.exe" start-server
```

Then when u do this :

```
& "C:\Users\shivli\AppData\Local\Android\Sdk\platform-tools\adb.exe" devices
```

>> List of devices attached

emulator-5554 device

This means emulator running in background. So on every fresh run you can try killing the daemon and restarting it again using above two commands

Step 1 : check jdk. (Jbr 17)

Step 2 : gradle-wrapper.properties

```
distributionBase=GRADLE_USER_HOME  
distributionPath=wrapper/dists  
  
distributionUrl=https\://services.gradle.org/distributions/gradle-8.13-bin.zip  
networkTimeout=10000  
validateDistributionUrl=true  
zipStoreBase=GRADLE_USER_HOME  
zipStorePath=wrapper/dists
```

Step 3 : gradle/libs.versions.toml

```
[versions]  
agp = "8.1.0"  
junit = "4.13.2"  
junitVersion = "1.1.5"  
espressoCore = "3.5.1"  
appcompat = "1.6.1"  
material = "1.9.0"  
activity = "1.7.2"  
constraintlayout = "2.1.4"  
  
[libraries]  
junit = { group = "junit", name = "junit", version.ref = "junit" }  
ext-junit = { group = "androidx.test.ext", name = "junit", version.ref =  
"junitVersion" }  
espresso-core = { group = "androidx.test.espresso", name = "espresso-core",  
version.ref = "espressoCore" }  
appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref =  
"appcompat" }  
material = { group = "com.google.android.material", name = "material",  
version.ref = "material" }  
activity = { group = "androidx.activity", name = "activity", version.ref =  
"activity" }  
constraintlayout = { group = "androidx.constraintlayout", name =  
"constraintlayout", version.ref = "constraintlayout" }
```

```
[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
```

Step 4 : app/build.gradle

```
plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace 'com.example.myapplication'
    compileSdk 34

    defaultConfig {
        applicationId "com.example.myapplication"
        minSdk 24
        targetSdk 34
        versionCode 1
        versionName "1.0"

        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles
getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_11
        targetCompatibility JavaVersion.VERSION_11
    }
}

dependencies {
    implementation libs.appcompat
    implementation libs.material
    implementation libs.activity
    implementation libs.constraintlayout
    testImplementation libs.junit
    androidTestImplementation libs.ext.junit
    androidTestImplementation libs.espresso.core
    configurations.all {
        exclude group: "org.jetbrains.kotlin", module: "kotlin-stdlib-jdk7"
    }
}
```

```

        exclude group: "org.jetbrains.kotlin", module: "kotlin-stdlib-jdk8"
    }
}
configurations.all {
    exclude group: "org.jetbrains.kotlin", module: "kotlin-stdlib-jdk7"
    exclude group: "org.jetbrains.kotlin", module: "kotlin-stdlib-jdk8"
}

```

Make activities and dont forget to add them in manifests

If you get the error R wala - then check package - myapplication thingy

Intents are used to navigate from one activity to another

TYPES OF INTENTS :

1. Explicit Intent (YOU specify target)

👉 Used most in your labs

```

Intent intent = new Intent(ThirdActivity.this, FourthActivity.class);
startActivity(intent);

```

- ✓ You KNOW exactly where to go
- ✓ Used for navigation inside your app

2. Implicit Intent (Android decides)

👉 You just say *what* you want

```

Intent intent = new Intent(Intent.ACTION_VIEW);
intent.setData(Uri.parse("https://www.google.com"));
startActivity(intent);

```

You have one xml for each page on your app

Basically xml file defines what the ui for each page (what layout? What contents of the layout? What type of buttons and shi)

Then java Main.java provides functionality/logic to those UI elements

It is the bridge between xml and java

An xml looks like this

```
<LinearLayout>
    <Button />
    <TextView />
</LinearLayout>
```

Layouts? (containers where UI elements are kept, layout styles are the styles in which you want your UI elements to be arranged)

Linear Layout : arrange items in line either horizontal or vertical

Relative layout : position items relative to each other

Constraint layout : attach elements to the edges of the screen (button in middle, text up, image down)

Frame layout : to stack elements on top of each other

Table layout : like a table

Understand hierarchy. Layouts can contain layouts. They're containers after all

```
<LinearLayout>
```

```
    <TextView />
```

```
    <LinearLayout>
```

```
        <Button />
```

```
        <Button />
```

```
    </LinearLayout>
```

```
</LinearLayout>
```

EVERY view in XML MUST have:

android:layout_width

android:layout_height

ADAPTORS IN LAYOUTS?

It takes data (array/list) → converts it into views (UI items)

Where do we use Adapters?

Whenever you have **repeating items**:

- ✓ Contact list
- ✓ Messages
- ✓ Instagram feed
- ✓ Menu list

Basically for example if i have a list to display on click of a button ill keep all contents as an array and then use an adaptor to arrange all these into a view (listview preferably)

Binds data to views

Used for fetching data from lists/ databases/arrays

ADAPTOR WITH LISTVIEW, GRIDVIEW, WITH ARRAY ADAPTOR

A. Adapter with ListView:

The ListView uses an adapter to display data in a vertically scrollable list. The adapter manages the rows in the list by binding data to them.

Using ArrayAdapter with ListView.:

```
ListView listView = findViewById(R.id.listView);
String[] items = {"Item 1", "Item 2", "Item 3"};
ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
android.R.layout.simple_list_item_1, items);
listView.setAdapter(adapter);
```

B. Adapter with GridView:

The GridView displays data in a grid format. An adapter provides the data for each cell in the grid.

Using ArrayAdapter with GridView:

```
GridView gridView = findViewById(R.id.gridView);
String[] items = {"A", "B", "C", "D", "E"};
ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
android.R.layout.simple_list_item_1, items);
gridView.setAdapter(adapter);
```

TABLAYOUT - TO SWITCH BTW LAYOUTS TABBB

TabLayout is often used with a ViewPager or ViewPager2 to display and navigate content dynamically.

SPINNERS AND PICKERS

1. SPINNER (Dropdown list)

What is it?

A **Spinner** is basically a **dropdown menu** in Android.

 User clicks → sees list → selects ONE option

SPINNER ALSO USES AN ADAPTOR TO SHOW DATA

2. PICKERS

Pickers are used to **select values like date, time, number etc.**



(A) DatePicker

Used for:

- DOB selection
- Booking dates

Or timepicker for alarm

Adding images in Spinners can enhance the user interface, especially when dealing with options that are better represented visually (e.g., flags for countries or product images).

How?

Steps to Add Images in Spinners

1. **Create a Custom Layout for Spinner Items** Design a custom XML layout for each item in the Spinner. This layout can include an ImageView alongside a TextView.

Example: `spinner_item.xml`

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:padding="8dp">
```

32

```
    <ImageView
        android:id="@+id/image"
        android:layout_width="40dp"
        android:layout_height="40dp"
        android:layout_marginEnd="8dp" />

    <TextView
        android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="16sp"
        android:textColor="#000" />
</LinearLayout>
```

2. **Prepare the Data** Create a data class to hold the text and image resource for each item.

Example: Data Class

```
public class SpinnerItem {  
    private String text;  
    private int imageResId;  
  
    public SpinnerItem(String text, int imageResId) {  
        this.text = text;  
        this.imageResId = imageResId;  
    }  
    public String getText() {  
        return text;  
    }  
    public int getImageResId() {  
        return imageResId;  
    }  
}
```

3. **Create a Custom Adapter** Extend ArrayAdapter to bind the custom layout and data to the Spinner.

Example: CustomAdapter

```
public class CustomSpinnerAdapter extends ArrayAdapter<SpinnerItem> {  
    public CustomSpinnerAdapter(Context context, List<SpinnerItem> items) {  
        super(context, 0, items);  
    }  
    @NonNull  
    @Override  
    public View getView(int position, @Nullable View convertView, @NonNull  
    ViewGroup parent) {
```



```

        return createView(position, convertView, parent);
    }
    @Override
    public View getDropDownView(int position, @Nullable View convertView,
    @NonNull ViewGroup parent) {
        return createView(position, convertView, parent);
    }
    private View createView(int position, View convertView, ViewGroup parent)
    {
        if (convertView == null) {
            convertView =
            LayoutInflater.from(getContext()).inflate(R.layout.spinner_item, parent, false);
        }
        SpinnerItem currentItem = getItem(position);
        ImageView imageView = convertView.findViewById(R.id.image);
        TextView textView = convertView.findViewById(R.id.text);

        if (currentItem != null) {
            imageView.setImageResource(currentItem.getImageResId());
            textView.setText(currentItem.getText());
        }
        return convertView;
    }
}

```

4. **Bind the Adapter to the Spinner** Populate the Spinner with a list of SpinnerItem objects and attach the custom adapter.

Example: MainActivity

```

Spinner spinner = findViewById(R.id.spinner);
List<SpinnerItem> items = new ArrayList<>();
items.add(new SpinnerItem("Apple", R.drawable.apple));
items.add(new SpinnerItem("Banana", R.drawable.banana));
items.add(new SpinnerItem("Cherry", R.drawable.cherry));

CustomSpinnerAdapter adapter = new CustomSpinnerAdapter(this, items);
spinner.setAdapter(adapter);

```

4. **Bind the Adapter to the Spinner** Populate the Spinner with a list of SpinnerItem objects and attach the custom adapter.

Example: MainActivity

```
Spinner spinner = findViewById(R.id.spinner);
List<SpinnerItem> items = new ArrayList<>();
items.add(new SpinnerItem("Apple", R.drawable.apple));
items.add(new SpinnerItem("Banana", R.drawable.banana));
items.add(new SpinnerItem("Cherry", R.drawable.cherry));

CustomSpinnerAdapter adapter = new CustomSpinnerAdapter(this, items);
spinner.setAdapter(adapter);

spinner.setOnItemSelectedListener(new
AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parent, View view, int position,
long id) {
```

34

OBILE APPLICATION DEVELOPMENT LAB MANUAL

```
SpinnerItem selectedItem = (SpinnerItem)
parent.getItemAtPosition(position);
Toast.makeText(getApplicationContext(), "Selected: " +
selectedItem.getText(), Toast.LENGTH_SHORT).show();
}

@Override
public void onNothingSelected(AdapterView<?> parent) {
    // Handle the case where no item is selected
}
});
```

When do you make a new java class? Also dont forget to add the activity class to manifest